



Sistemi UAV: autopilota PX4 e sistema operativo NuttX

Corso di Sistemi Operativi

Ing. Arnaldo Tomasino

Ingegneria Informatica e dell'Automazione
(A.A. 2021/2022)

Indice

- Sistemi UAV
- Autopilota PX4 → *USATO NEL CONTROLLO DEI DRONI*
 - architettura
 - caricamento
 - integrazione moduli esterni
- Software di simulazione
- QGroundControl
- Sistemi operativi real-time e O. S. NuttX → *USA PX4*
 - funzionamento e architettura
 - file system
 - sistema grafico e NuttShell

Sistemi UAV

- **Unmanned Aerial Vehicle** anche detto "drone"
 - aeromobile senza pilota a bordo
 - pilotato da remoto con l'ausilio di un **autopilota** e di una **Ground Control Station (GCS)** oppure con traiettoria di volo pre-impostata
↳ A TERRA SI CONTROLLA IL MOVIMENTO
- **Ground Control Station:** sistema di terra che permette il controllo di un UAV da remoto
 - pilotaggio e modifica dell'assetto in tempo reale
 - controllo dei parametri di volo e dei dati dai sensori di bordo
↳ DATI, FOTOGRAFIE, MISURAZIONI...
- **autopilota** → CONTROLLA IL MOVIMENTO E RICEVE INFORMAZIONI DA TERRA
 - dispositivo elettronico dalle piccole dimensioni che consente il controllo dei singoli motori
 - attua le direttive di missione ricevute dalla GCS
 - regola attuatori e motori al fine di garantire un assetto sicuro durante il volo

Sistemi UAV (2)

- classificati in base a

- HARDWARE
DIVERGENTI*
- raggio operativo
 - quota di volo
 - durata del volo
 - peso massimo al decollo

- ambiti di utilizzo

- militare
- agricoltura
- vigilanza
- riprese fotografiche e cinematografiche
- missioni di ricerca

- affidabilità in volo garantita da micro-dispositivi di bordo (sensori)

- GARANIZIONE
AFFIDABILITÀ
IN VOLO
SENSORI*
- di posizione e orientamento (accelerometro, giroscopio, magnetometro)
 - di telerilevamento (GPS)
 - ambientali (temperatura e umidità, barometri, controllo della qualità dell'aria)
- ↳ sensori minimi*



Autopilota PX4



→ SONO LIMITATI DALLE CARATTERISTICHE HARDWARE

- Un autopilota comprende l'insieme delle routine SW per il governo di un UAV senza intervento da parte di operatori umani

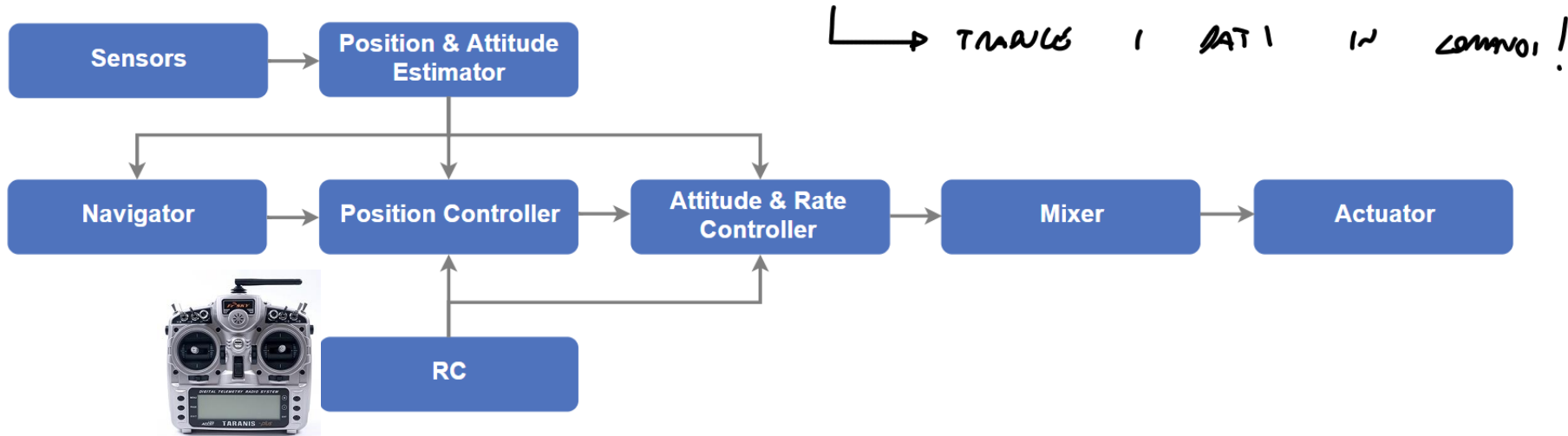
- PX4: software di controllo open-source per sistemi UAV

- distribuito con licenza BSD → FREE MA NON IMBONE LA PRODUZIONE DELLE MODIFICHE!
- sviluppo iniziato nel 2009
- suddiviso in due layer principali: DIFFERENZA DA OPEN SOURCE DI LINUX
 - flight stack
 - stack di moduli software per il controllo dell'hardware
 - comprende algoritmi di navigazione e controllo per aeromobili
 - fixed-wing, VTOL (Vertical Take-Off and Landing) e multirottore
 - middleware

↳ COMUNICAZIONE CON L'ESTERNO E INTEGRAZIONE HARDWARE!

Architettura: flight stack

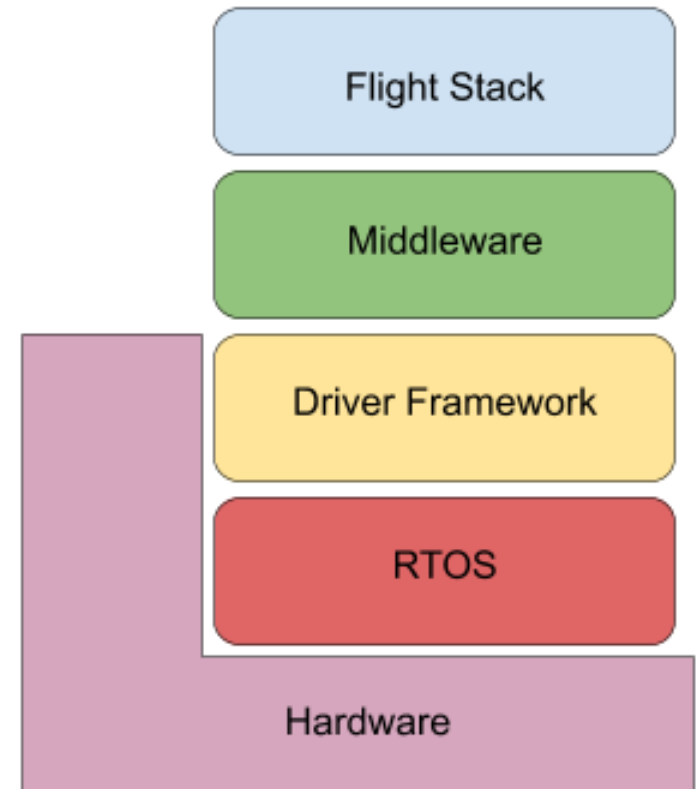
- **Estimator**: riceve dati dai sensori a bordo per definire lo stato attuale del velivolo (STIMA I PARAMETRI A QUESTIONE DEL VEICOLO)
 - e.g., assetto dai **dispositivi inerziali** (accelerometro e giroscopio), **BAROMETRO**
- **Controller**: prende come input la misura dello stato stimato per effettuare correzioni sul veicolo e raggiungere il *setpoint*
 - e.g., *position controller*: regolazione assetto e spinta per raggiungere posizione desiderata, definita dai *position setpoints* (calcolare ciò che è necessario per il movimento)
- **Mixer**: traduce i dati elaborati dai controllori in comandi verso i motori assicurandosi che non vengano superati i limiti strutturali



Architettura: middleware

Composto da:

- driver per i sensori integrati a bordo
- moduli per la comunicazione
 - *uORB*: API per lo scambio di messaggi tra *thread* o tra processi
 - paradigma *publish/subscribe* asincrono
 - *MAVLink*: protocollo leggero per
 - comunicazione tra velivolo e ground control station
 - scambio di messaggi tra autopilota e altri componenti esterni
- layer di simulazione → CONTROLLATO DAL S.O.



Caricamento del firmware

- repository contenente il firmware di PX4 presente su GitHub (<https://github.com/PX4/PX4-Autopilot>)
- generazione degli eseguibili a partire dal codice sorgente tramite cross-compilazione
 - utilizzo del tool *make*
 - i *makefile* definiscono le regole per la costruzione dei moduli caricabili (sorgenti da includere, librerie da linkare)

```
make px4_fmu-v2_default
```

- in questo caso viene compilato il target *px4 fmu-v2*: versione del firmware da compilare per l'hardware utilizzato
- *default* indica la configurazione scelta

► COMPILAZIONE SU SISTEMI AVERSI E CARICAMENTO SU AERU !

- per includere su una piattaforma il codice compilato

```
make px4_fmu-v2_default upload
```


Integrazione di moduli esterni

px4 non essere integrati in ROS

- firmware PX4 estensibile con altri moduli
- inclusione applicazioni esterne in directory *src/examples*
 - indicazione delle applicazioni da includere nella compilazione in *default.cmake* nella directory */boards/px4/fmu-v2*
 - file *CMakeLists.txt* con direttive per la compilazione del codice aggiunto
- possibilità di rendere il software dell'autopilota intelligente includendo sistemi di ragionamento automatico su uno scenario modellato in una base di conoscenza

↓
MACHINE LEARNING BASE PER LA LIMITAZIONE HARDWARE

Software di simulazione

○ Gazebo

- ambiente di simulazione 3D per qualsiasi robot
- GUI completa per la personalizzazione e il controllo
- Possibilità di creare nuovi mondi o modelli (implementati in un formato XML specifico)
- adatto alla simulazione di algoritmi di *computer vision* e di *object-avoidance*
- simulazione multi-veicolo
- spesso utilizzato con ROS

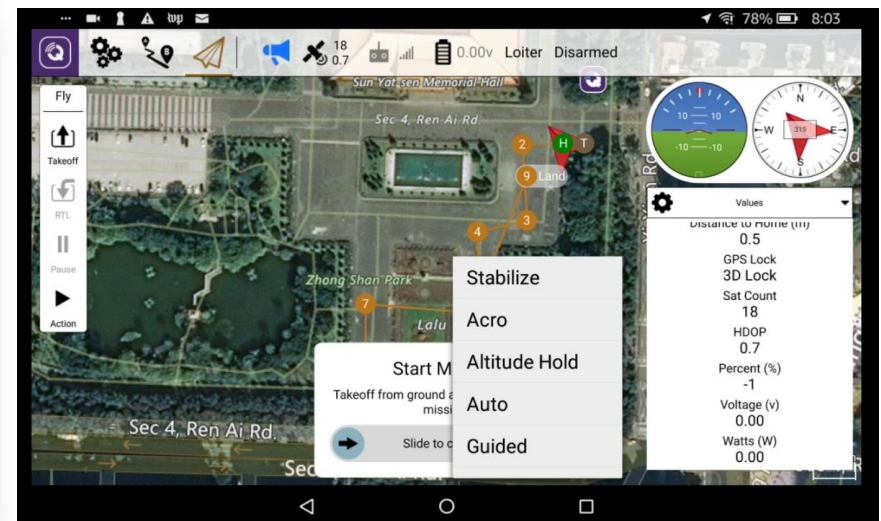
○ jMAVSim (SOFTWARE BASICO)

- Java3D per l'implementazione grafica
- controllabile tramite ROS, QGroundControl (tramite UDP)
- impossibile integrare sensori ulteriori o ostacoli
- nessuna ispezione visuale

QGroundControl

Applicazione per configurare e guidare un velivolo con autopilota PX4

- mission planning per voli autonomi
- visualizzazione di mappe per mostrare la posizione del velivolo
- supporto per la gestione di più velivoli
- gestione dei sensori e dei relativi dati in tempo reale
- adatto a piattaforme Windows, OS X, Linux, iOS e Android



Sistemi operativi real-time (TEMP REAL)

- Classe di sistemi operativi destinati ad applicazioni real-time
 - il sistema dev'essere **deterministico** → altrimenti l'operazione fallisce
 - risposta da restituire entro un certo intervallo di tempo (**deadline**)
 - tempi di esecuzione subordinati alla certezza di esecuzione
- Due tipi di sistemi real-time
 - *Hard real-time*: (molto sensibili)
 - *deadline* sui processi
 - esempi: sistemi di controllo su automobili, avionica, sistemi di controllo di processi nucleari
 - *Soft real-time*: (più permissivi)
 - ammettono modeste violazioni di talune *deadline*
 - la violazione delle *deadline* influenza la QoS (*Quality of Service*)
 - esempio: sistemi multimediali
- Esecuzione basata su **eventi**:
 - appena un evento si verifica, il sistema deve rispondere prima possibile

Scheduling negli RTOS

○ Round-robin scheduling

- algoritmo con **preemption** *Time slice*
- definizione di quanti di tempo in cui un processo può usare la CPU
- potrebbe portare alla violazione delle *deadline* previste

○ Priority-based scheduling

- priorità alta a processi real-time
- algoritmo anche con *preemption*
- adatto solo a sistemi soft real-time
- non tiene conto delle **deadline** previste

○ Rate-monotonic scheduling

- priorità **statica** in base al periodo di utilizzo della CPU:
più alta priorità al processo che richiede la CPU più spesso
- basato sull'attivazione della *preemption*
- adatto a sistemi hard-real time (*NuttX*)
consente il rispetto delle *deadline* previste
rende deterministico il comportamento dei processi

contra anche l'utilizzo della CPU

Sistema operativo NuttX



Lo si basa sull'autopilota PX4 REAL TIME

- Sistema operativo real-time rilasciato nel 2007
 - creato da Gregory Nutt
 - open-source secondo licenza BSD
- Scalabile per microcontrollori da 8-bit a 32 bit
- Scritto prevalentemente in linguaggio C
- Basato sulle API standard POSIX e ANSI
- Ampio uso delle API Unix
 - NuttShell: interprete di comandi Linux
 - (Virtual) File System
- Kernel estensibile con moduli esterni
- Supporto per i protocolli di comunicazione dell'IoT
- Piattaforme supportate
 - ARM, OpenRisc, Intel x8086, Atmel AVR, RISC-V, Zilog

*NO MEMORIA DI
MASSA !*

NuttX in prodotti industriali



- Registratori vocali Sony
- Microcontrollori Sony Spresense
- Motorola MotoMod
 - NuttX per gestire la comunicazione tra smartphone e funzionalità del modulo aggiuntivo
- Samsung TizenRT
 - sistema Real-Time basato su kernel NuttX
 - adatto a Smart TV, dispositivi indossabili, elettrodomestici *smart home*



Platform Extension

TIZEN
RT

Web applications

JerryScript/IoT.js

IoTivity (OCF)

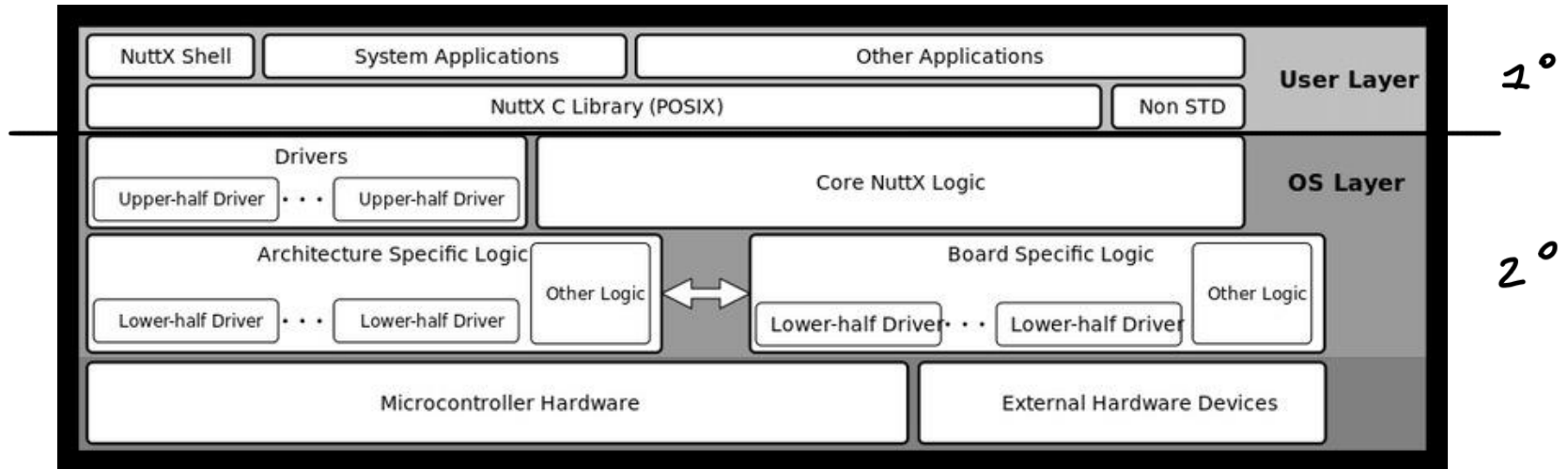
Lightweight RTOS
(TinyAra)



ARTIK 1

Entry-level
Home Appliance

Architettura di NuttX



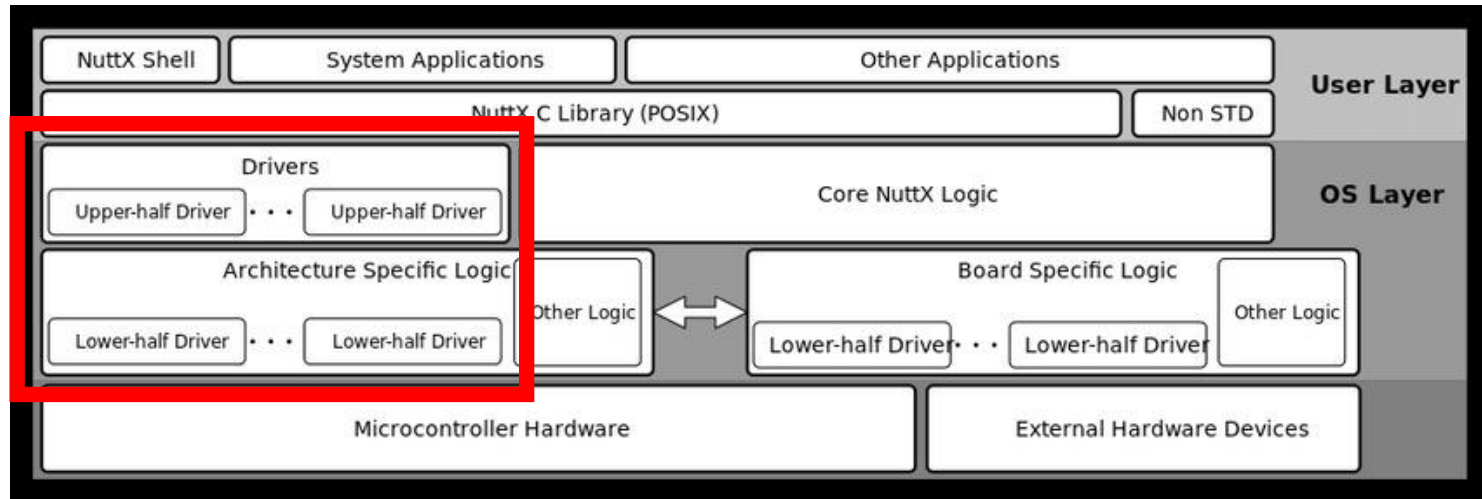
○ User layer

- layer di comunicazione con l'utente
 - shell, applicazioni esterne e script
- layer di librerie POSIX
 - permettono la comunicazione delle applicazioni con i meccanismi implementati nel layer inferiore

○ OS Layer

- logica di base del sistema operativo NuttX (algoritmi di scheduling, gestione risorse, meccanismi IPC)
- driver per i possibili hardware

Architettura di NuttX (2)



- **Upper-half driver**
 - espongono interfacce indipendenti dall'hardware verso lo *user layer*
 - implementazione di operazioni generiche (*read, write*)
 - tramite invocazione dei *lower-half driver*
- **Lower-half driver**
 - implementazione dipendente dall'hardware
- Librerie organizzate tramite le directory che compongono l'implementazione del sistema operativo

Task in NuttX



↳ GESTIONE DEI PROCESSI

- equivalente di un processo: entità dinamica che rappresenta un programma in esecuzione
- a differenza dei processi classici, non gestiscono uno spazio di indirizzi privato
 - molti microcontrollori non possiedono *Memory Management Unit*
 - NuttX opera solo con un *flat address space*
- descritti dalla struttura dati chiamata Task Control Block (TCB)
 - insieme delle risorse associate (variabili d'ambiente, *stream*, *file descriptor*)
 - stato nel campo *task_state*
- identificato da un *taskID*
 - dato il *taskID*, il sistema operativo può reperire il TCB e viceversa
- organizzati tramite relazioni *parent-child*
 - *child*: eredita i *file stream*, *stdin/stdOUT*
 - *parent*: attende la terminazione dei processi figli per essere rimosso

↳ PROCESSI PADRE CHE GENERANO I PROCESSI FIGLI

IPC in NuttX

SINCRONIZAZIONE TRA PROCESSI !

○ Signals

- qualsiasi task o l'*interrupt handler* può direttamente inviare un segnale ad un altro task (tramite *taskID*)
- il task svegliato esegue una specifica funzione di gestione dell'*interrupt*

○ POSIX Message Queue

- ad ogni task è assegnata una coda in cui il *main* rimane in attesa di messaggi
- l'*interrupt handler* può inviare messaggi verso una coda quando si verificano eventi
- appena un messaggio arriva, può svegliare il task e innescare l'elaborazione specifica

○ Semafori

- *mutual exclusion*
- *signaling semaphores*

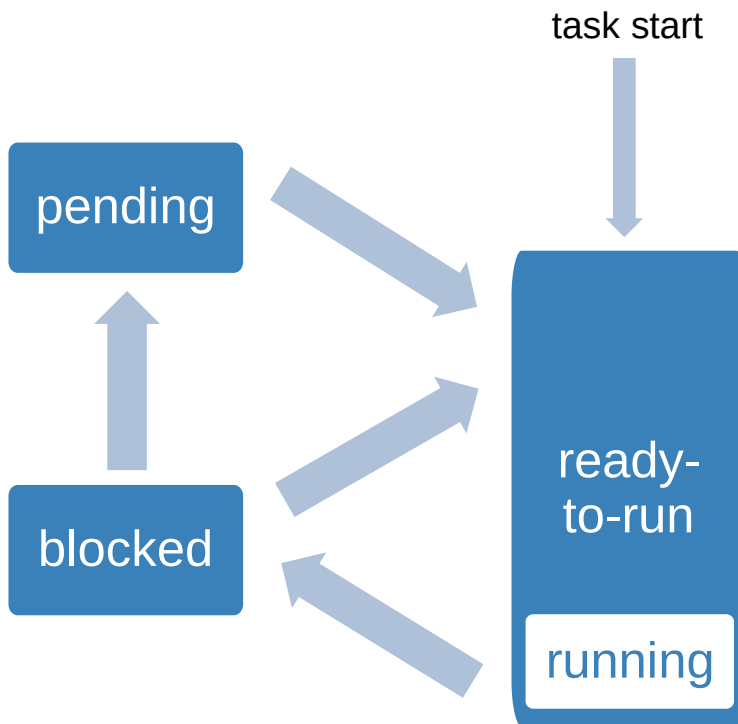
TASK A	TASK B
<code>sem_init(sem, 0, 0);</code>	
<code>sem_wait(sem);</code>	
<i>blocked</i>	
	<code>sem_post(sem);</code>
<i>_Awakens as holder _</i>	

TASK A	TASK B
<code>sem_wait(sem);</code>	
<i>have access</i>	
<i>priority boost</i>	<code>sem_wait(sem);</code>
	<i>blocked</i>
<code>sem_post(sem);</code>	
<i>priority restored</i>	<i>have access</i>

Scheduling in NuttX



- Lo stato di un task è indicato anche dalla categoria a cui appartiene



- `inactive_tasks`

- lista di tutti i task inizializzati ma non ancora attivati

- `readytorun`

- lista dei task pronti a partire
- testa: task attivo corrente
- coda: task in idle

- `pendingtasks`

- task pronti per partire ma
 - bloccati perché per il processo in esecuzione è disabilitata la *preemption*
 - hanno priorità minore rispetto al processo in testa alla lista *readytorun*

- liste riferite allo stato *blocked*

- `waitingforsemaphore`
- `waitingforsignal`

OGNI TASK HA UNA STRUTTURA SPECIFICA:

File system NuttX



non è obbligatorio in NuttX

- **Pseudo file-system**

- non richiesto per il funzionamento
- è semplicemente una struttura dati ad albero in RAM
- non richiede dispositivi di storage fisici

(NESSUNA
MEMORIA
SICERAMENTE!)

- se abilitato, parte da un pseudo path *root* (/)
- estensibile: altri file system reali possono essere montati a partire dalla sua struttura
 - similmente a Linux, file speciali dei dispositivi nella directory */dev*
 - comando *mount* per montare il FS dei dispositivi nella directory */mnt*
- implementazione del FS realizzata con le interfacce dello standard POSIX
 - funzioni `open()`, `close()`, `read()`, `write()`

Sottosistema grafico NX

NuttX possiede anche un'interfaccia minimale

- **sistema client/server** simile ad X-Window
 - client: richiede al server la rappresentazione degli oggetti grafici
 - server: back-end NX
 - primitive per le operazioni grafiche e di rendering
 - interfacciamento con l'utente
- finestre disposte in uno spazio verticale virtuale
 - finestre attive e inattive
- API per il supporto ai dispositivi di puntamento
 - NX fornisce gli input alle callback del client
- supporto completo per tastiera
- **NxWidgets**: sottosistema per l'integrazione di oggetti grafici ("widget") all'interno di NX
 - prevalentemente scritto in C++

Sottosistema grafico NX - GUI



NuttShell

- Shell di riferimento per NuttX
- basata sulla shell BASH
- formati di comandi supportati: *COMANDI SIMILI A LINUX!*
 - `<cmd>`
 - `<cmd> > <file>`
`<cmd> >> <file>`
 - `<cmd> &`
- variabili built-in
 - PWD e OLDPWD
 - `$?` per il risultato dell'esecuzione dell'ultimo comando
- comandi principali dal sistema operativo Linux:
 - `pwd` e `cd` per mostrare directory corrente e cambiare directory
 - `ls` per listare le directory
 - `mount` per montare un file system esterno sul pseudo file-system
 - comandi per la gestione della rete (es.: `ifconfig`, `ping`)
 - esecuzione di script esterni con `sh`

NuttShell

- comportamento di default all'avvio

- NSH crea un file system minimale

```
'--init.d/  
    '- rcS
```

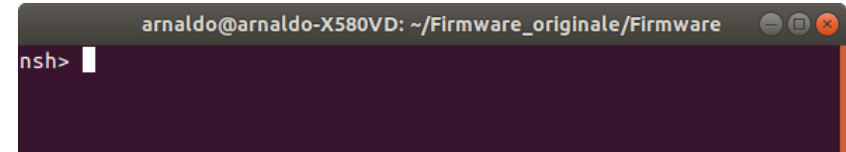
- NSH monta il file system creato in */etc*

```
|--dev/  
|    '-- ram0  
|--etc/  
    '--init.d/  
        '- rcS
```

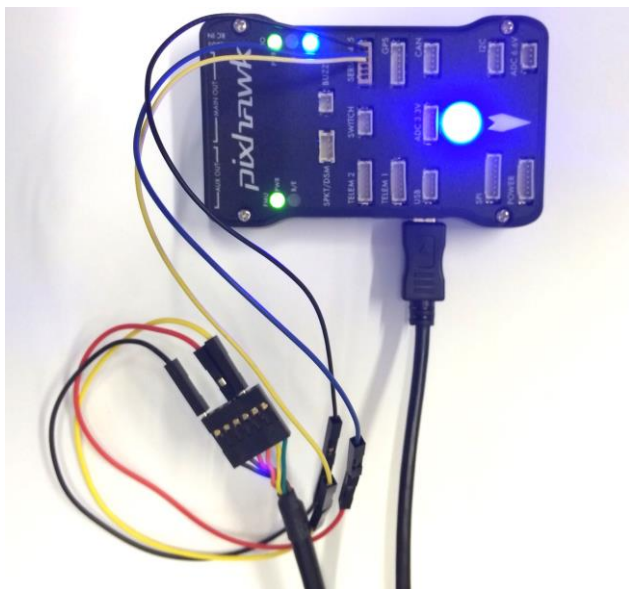
- NSH esegue lo script */etc/init.d/rcS* prima del primo prompt. Lo script modifica il file system root in questo modo:

```
|--dev/  
|    |-- ram0  
|    '-- ram1  
|--etc/  
|    '--init.d/  
|        '-- rcS  
|--tmp/
```

- parametri di configurazione nel file *defconfig*



Operazioni in NuttShell



ESEMPIO BOARD DI UN
DRONE (PX4-NUITX)

- comunicazione con *board* tramite il *serial client* Screen

```
screen /dev/ttyXXX BAUDRATE 8N1
```

```
arnaldo@arnaldo-X580VD: ~/Firmware_originale/Firmware
nsh> help
help usage:  help [-v] [<cmd>]

[      cp      export  ls      mw      set      true
?      date    false   mkdir  ps      sh      umount
break  echo     free   mkfatfs pwd    sleep  unset
cat    exec    help   mount  rm      test   usleep
cd     exit    kill   mv     rmdir   time

Builtin Apps:
l3gd20      airmass_selector  fw_pos_control_l1  hmc5883
uorb        reboot            mc_att_control     sensors
ms5611      rc_update         top                 serdis
load_mon    ekf2              mavlink             mc_rate_control
ms4525_airspeed  navigator         nsh                 px4io
vtol_att_control  adc               mixer               fw_att_control
gps            lsm303d           logger              perf
fmu            sercon            mtd                 pwm
commander      bl_update         land_detector       testMinime
mpu6000        param             dataman             mc_pos_control
nsh>
```

```
nsh> free
          total      used      free      largest
Umem:    239008     151824     87184     87136
nsh>
```

